

ONBRD-08: Your First Queries

Series: ONBRD — Dynatrace Onboarding | **Notebook:** 8 of 10 | **Created:** December 2025 | **Last Updated:** 10/02/2026

Learning DQL Fundamentals

Dynatrace Query Language (DQL) is how you access data in Grail. This notebook introduces the core concepts and patterns you'll use daily.

Table of Contents

1. [DQL Basics](#)
 2. [The Pipeline Model](#)
 3. [Fetching Data](#)
 4. [Filtering](#)
 5. [Selecting Fields](#)
 6. [Aggregating with Summarize](#)
 7. [Sorting and Limiting](#)
 8. [Time Ranges](#)
 9. [Common Patterns](#)
 10. [Next Steps](#)
-

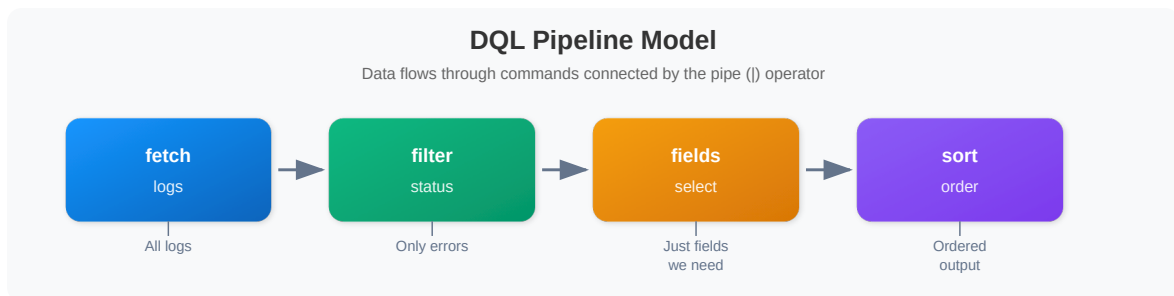
Prerequisites

- Dynatrace environment with data (ONBRD-05, ONBRD-06, ONBRD-07)
- DQL query permissions

- Access to Notebooks or the DQL query interface

1. DQL Basics

DQL is a **pipeline-based query language**—not SQL. Data flows through a series of commands connected by the pipe (`|`) operator.



DQL vs SQL

DQL	SQL	Note
<code>fetch logs</code>	<code>SELECT * FROM logs</code>	Start with data source
<code>\ filter x == "y"</code>	<code>WHERE x = 'y'</code>	Use <code>==</code> , double quotes
<code>\ fields a, b</code>	<code>SELECT a, b</code>	Field selection after fetch
<code>\ summarize count()</code>	<code>SELECT COUNT(*)</code>	Aggregation command
<code>by: {field}</code>	<code>GROUP BY field</code>	Grouping syntax
<code>{"a", "b"}</code>	<code>('a', 'b')</code>	Array syntax (curly braces)

2. The Pipeline Model

Each command in the pipeline operates on the output of the previous command:

```
fetch logs, from:-1h           // 1. Get the last hour of logs (time range
on fetch)
| filter status == "ERROR"     // 2. Keep only error-class records
| fields timestamp, content    // 3. Select columns
| sort timestamp desc         // 4. Order by time
| limit 100                   // 5. Take first 100
```

Order Matters

- **Filter early** - Reduces data before expensive operations
- **Select fields** - Reduces memory usage
- **Aggregate** - Summarize before sorting
- **Sort** - Order the final results
- **Limit** - Control output size

3. Fetching Data

Every DQL query starts with a `fetch` command specifying the data source.

```
// Fetch logs (most common)
fetch logs, from:-1h
| limit 10
```

```
// Fetch spans (distributed traces)
fetch spans, from:-1h
| limit 10
```

```
// Fetch entity data (hosts)
fetch dt.entity.host
| limit 10

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | limit 10
// Caveat: Smartscape can report fewer entities than the classic entity store,
// and
// both return only entities seen in the query timeframe (default 2 h) - for a
// discovery inventory add a timeframe such as from:-7d to either query.
```

```
// Fetch problems
fetch dt.davis.problems, from:-24h
| limit 10
```

Common Data Sources

Source	Description
logs	Log records
spans	Distributed trace spans
events	Davis, Kubernetes, custom, and other platform events (self-monitoring is <code>dt.system.events</code>)
bizevents	Business events
dt.entity.host	Host entities
dt.entity.service	Service entities
dt.entity.process_group	Process group entities
dt.davis.problems	Detected problems

4. Filtering

Use `filter` to narrow results. Filter as early as possible for performance.

loglevel vs status on logs. `loglevel` holds the detected severity — `ERROR` , but also `SEVERE` , `CRITICAL` , `ALERT` , `EMERGENCY` , and so on. Dynatrace adds a derived `status` field that groups `SEVERE` , `ERROR` , `CRITICAL` , `ALERT` , `FATAL` , and `EMERGENCY` into `ERROR` . To find every error-class record, filter `status == "ERROR"` : on one tenant over one hour, `loglevel == "ERROR"` matched about 64,000 records and `status == "ERROR"` about 118,000. Both fields use uppercase values.

Sources: [Automatic log enrichment \(DT docs\)](#) — “a status attribute is created with a value that is a sum of loglevel values”.

```
// Filter by equality - loglevel values are UPPERCASE ("ERROR", "WARN",
"INFO");
// a lowercase "error" matches nothing and raises no error. loglevel is the
raw
// detected level (SEVERE, CRITICAL, ... are separate values); to catch every
// error-class record use the derived field instead: status == "ERROR"
fetch logs, from:-1h
| filter loglevel == "ERROR"
| limit 20
```

```
// Filter with multiple conditions (AND)
fetch logs, from:-1h
| filter status == "ERROR" and contains(content, "timeout")
| limit 20
```

```
// Filter with or condition
fetch logs, from:-1h
| filter status == "ERROR" or status == "WARN"
| limit 20
```

```
// Filter using IN for multiple values
fetch logs, from:-1h
| filter in(status, {"ERROR", "WARN"})
| limit 20
```

```
// Filter with string matching
fetch logs, from:-1h
| filter contains(content, "timeout")
| limit 20
```

Filter Operators

Operator	Example	Description
<code>==</code>	<code>field == "value"</code>	Equals
<code>!=</code>	<code>field != "value"</code>	Not equals
<code>></code> , <code><</code>	<code>count > 10</code>	Greater/less than
<code>>=</code> , <code><=</code>	<code>count >= 10</code>	Greater/less or equal

Operator	Example	Description
<code>and</code> , <code>or</code>	<code>a == 1 and b == 2</code>	Logical operators
<code>in()</code>	<code>in(field, {"a", "b"})</code>	Value in set
<code>contains()</code>	<code>contains(field, "text")</code>	Substring match
<code>isNull()</code>	<code>isNull(field)</code>	Field is null
<code>isNotNull()</code>	<code>isNotNull(field)</code>	Field is not null

5. Selecting Fields

Use `fields` to select specific columns. This improves readability and performance.

```
// Select specific fields from logs
fetch logs, from:-1h
| fields timestamp, loglevel, log.source, content
| limit 20
```

```
// Create calculated fields (duration / 1ms uses duration arithmetic –
preferred)
fetch spans, from:-1h
| fields span.name,
      duration,
      duration_ms = duration / 1ms
| limit 20
```

```
// Rename fields with aliases
fetch dt.entity.host
| fields name = entity.name,
        status = state,
        os = osType
| limit 20

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only field state, which has NO Smartscape node equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state detail. For monitoring mode, do not use
monitoringMode – it is
// empty on Kubernetes and Fargate hosts; read billing events instead (ONBRD-
05 § 6).
// Other fields do map: osType -> os.type (LINUX -> OS_TYPE_LINUX);
entity.name -> name.
```

fieldsAdd vs fields

Command	Effect
<code>fields</code>	Keeps only specified fields
<code>fieldsAdd</code>	Adds new fields, keeps all existing

```
// fieldsAdd keeps existing fields and adds new ones
fetch spans, from:-1h
| fieldsAdd duration_ms = duration / 1ms
| fields span.name, duration, duration_ms
| limit 10
```

6. Aggregating with Summarize

Use `summarize` to aggregate data. Combine with `by:` for grouping.

```
// Simple count
fetch logs, from: now() - 1h
| summarize total_logs = count()
```

```
// Count by group
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {loglevel}
| sort log_count desc
```

```
// Multiple aggregations
// dt.service.name is set on every span, whatever the ingest source
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize {
    request_count = count(),
    avg_duration = avg(duration),
    max_duration = max(duration)
}, by: {dt.service.name}
| sort request_count desc
| limit 20
```

```
// Conditional counting
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
}, by: {dt.service.name}
| fieldsAdd error_rate = 100.0 * errors / total
| sort error_rate desc
| limit 20
```

Common Aggregation Functions

Function	Description
<code>count()</code>	Count records
<code>countIf(condition)</code>	Count where condition is true
<code>sum(field)</code>	Sum values
<code>avg(field)</code>	Average value
<code>min(field)</code>	Minimum value
<code>max(field)</code>	Maximum value

Function	Description
<code>percentile(field, 95)</code>	95th percentile

7. Sorting and Limiting

Control output order and size.

```
// Sort descending (newest first)
fetch logs, from: now() - 1h
| fields timestamp, loglevel, content
| sort timestamp desc
| limit 20
```

```
// Sort by multiple fields
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize request_count = count(), by: {dt.service.name}
| sort request_count desc
| limit 10
```

```
// Find slowest spans
fetch spans, from: now() - 1h
| fields span.name, dt.service.name, duration
| sort duration desc
| limit 10
```

8. Time Ranges

Control the time range for your queries.

```
// Last hour (using the from: parameter on fetch)
fetch logs, from: now() - 1h
| summarize count()
```

```
// Specific time range (a one-hour window ending an hour ago)
fetch logs, from: now() - 2h, to: now() - 1h
| summarize count()
```

```
// Last 7 days
fetch dt.davis.problems, from: now() - 7d
| summarize problem_count = count(), by: {event.status}
```

Time Units

Unit	Example	Description
s	now() - 30s	Seconds
m	now() - 15m	Minutes
h	now() - 2h	Hours
d	now() - 7d	Days

9. Common Patterns

Here are patterns you'll use frequently.

Error Investigation

```
// Find error logs with context
fetch logs, from: now() - 1h
| filter status == "ERROR"
| fields timestamp, log.source, content
| sort timestamp desc
| limit 50
```

Service Performance

```
// Service response time summary (use duration / 1ms for ns→ms conversion)
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize {
    requests = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by: {dt.service.name}
| sort requests desc
| limit 20
```

Log Volume Analysis

```
// Log volume by source and severity
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {log.source, loglevel}
| sort log_count desc
| limit 30
```

Entity Inventory

```
// Host inventory with details
fetch dt.entity.host
| fields
  name = entity.name,
  state,
  os = osType,
  cores = cpuCores
| sort name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only field state, which has NO Smartscape node equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state detail. For monitoring mode, do not use
monitoringMode — it is
// empty on Kubernetes and Fargate hosts; read billing events instead (ONBRD-
05 § 6).
// Other fields do map: osType -> os.type (LINUX -> OS_TYPE_LINUX);
cpuCores -> cores; entity.name -> name.
```

10. Next Steps

With DQL fundamentals covered:

1. **ONBRD-09: Setting Up Alerts** — Configure alerting and notifications
2. Practice with your own data
3. Explore the DQL documentation for advanced functions
4. Try creating queries in Notebooks

Beyond the Basics

- **makeTimeseries** — convert event-based data (logs, spans, bizevents) into time-bucketed metric series. Different from the **timeseries** command, which queries pre-ingested metrics. Example: `fetch logs, from:-2h | makeTimeseries errors = countIf(status == "ERROR"), interval:5m, by:{k8s.cluster.name}` (an error count per 5 minutes)

- **Iterative array expressions** — `arrayAvg` , `arraySum` , `iAny` , etc. for working with timeseries arrays in `fieldsAdd` and `filter`
- **Smartscape topology navigation** — `smartscapeNodes` , `smartscapeEdges` , `traverse` for entity relationships (the modern alternative to `dt.entity.*`)

Where to Go Deeper

- **OPLOGS / OPMIG / OPIPE** — Log query patterns, OpenPipeline transformations
- **SPANS series** — Span-specific DQL patterns
- **DASH series** — Building dashboards from DQL queries
- **AIOPS series** — Davis problem queries, anomaly detection DQL

DQL Checklist

- ☐ Understand the pipeline model
- ☐ Can fetch different data types
- ☐ Can filter with conditions
- ☐ Can select and calculate fields (using duration arithmetic, not nanosecond constants)
- ☐ Can aggregate with summarize
- ☐ Can sort and limit results
- ☐ Can specify time ranges
- ☐ Know that `makeTimeseries` exists for event → series conversion

Summary

In this notebook, you learned:

- DQL uses a pipeline model (not SQL)
- `fetch` starts every query
- `filter` narrows results
- `fields` selects and calculates columns
- `summarize` aggregates data
- `sort` and `limit` control output

- Time ranges with `from:` and `to:`
 - Duration arithmetic (`duration / 1ms`) is preferred over nanosecond constants
-

References

- [DQL Reference](#)
 - [DQL Functions](#)
 - [DQL Commands](#)
 - [Dynatrace Query Language \(DT docs\)](#)
 - [Automatic log enrichment \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.